Failure/choice — pick the right tool — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: the real skill here isn't being able to write all four web layers — it's knowing which one a given job actually needs, so you don't put Django on a weekend demo, or reach for Streamlit on a real product that needs user logins. **If not:** you'd waste days building auth and an ORM for a throwaway demo, or hit a hard wall when a "quick Streamlit app" suddenly needs real users and proper routing.

The five scenarios

Scenario	Pick	Why (one sentence)
A hackathon demo of your agent, due tonight	**Streamlit**	A clickable UI in pure Python with zero web code — the fastest path from script to demo.
A public endpoint other apps will call with JSON	**FastAPI**	A typed, validated, auto-documented JSON API is exactly what code-callers want, and async handles slow LLM calls.
A small internal tool for your team	**Flask** (or Streamlit)	Flask when you want a real, custom web app you control; Streamlit if "a page with a box" is enough.
A customer product with logins and saved history	**Django**	Auth, a database ORM, migrations, and an admin panel come built in — weeks of work you don't write.
A chat interface for a support agent	**Streamlit** (`st.chat_*`)	`st.chat_message` / `st.chat_input` + `session_state` build a chatbot UI in a few lines.

The reasoning behind the picks

- **"Due tonight" / "demo" → Streamlit.** The question is speed-to-UI, and nothing beats a Python script that becomes a web page. You trade layout control and routing you don't need tonight.
- **"Other apps call it" / "JSON" → FastAPI.** The client is *code*, so you want a typed contract, automatic validation, and `/docs` — not an HTML page. Async is a bonus that matters for LLM latency.
- **"Small custom app you control" → Flask.** When you want real routes and your own HTML but none of Django's weight, Flask is the minimal, explicit middle ground.

- **"Logins and saved history" / "product" → Django.** The moment you need users, stored data, and someone to administer it, Django's batteries (auth, ORM, admin) stop being overkill and start saving you weeks.
- **"Chat" → Streamlit's chat widgets.** The chatbot UI is a solved problem in Streamlit; reach for it before hand-building a chat page.

The one rule: the AI pipeline is identical in all four — you choose by the **app around it** (speed, callers, users, data), never by the AI. Match the tool to the job, and the web layer stays the thin wrapper it should be.